Team 3 - Vulnerability & Risk Analysis Automation Platform

1. Objective

The objective is to build a fully automated vulnerability and risk analysis pipeline capable of:

- Collecting vulnerability findings from multiple scanners
- Correlating findings automatically
- Enriching vulnerabilities with threat intelligence
- Calculating intelligent risk scores
- Detecting exploitable attack paths
- Prioritizing remediation automatically
- Exporting results to visualization and SIEM

2. Problem Statement

Traditional vulnerability management systems generate thousands of alerts without proper prioritization.

Challenges include:

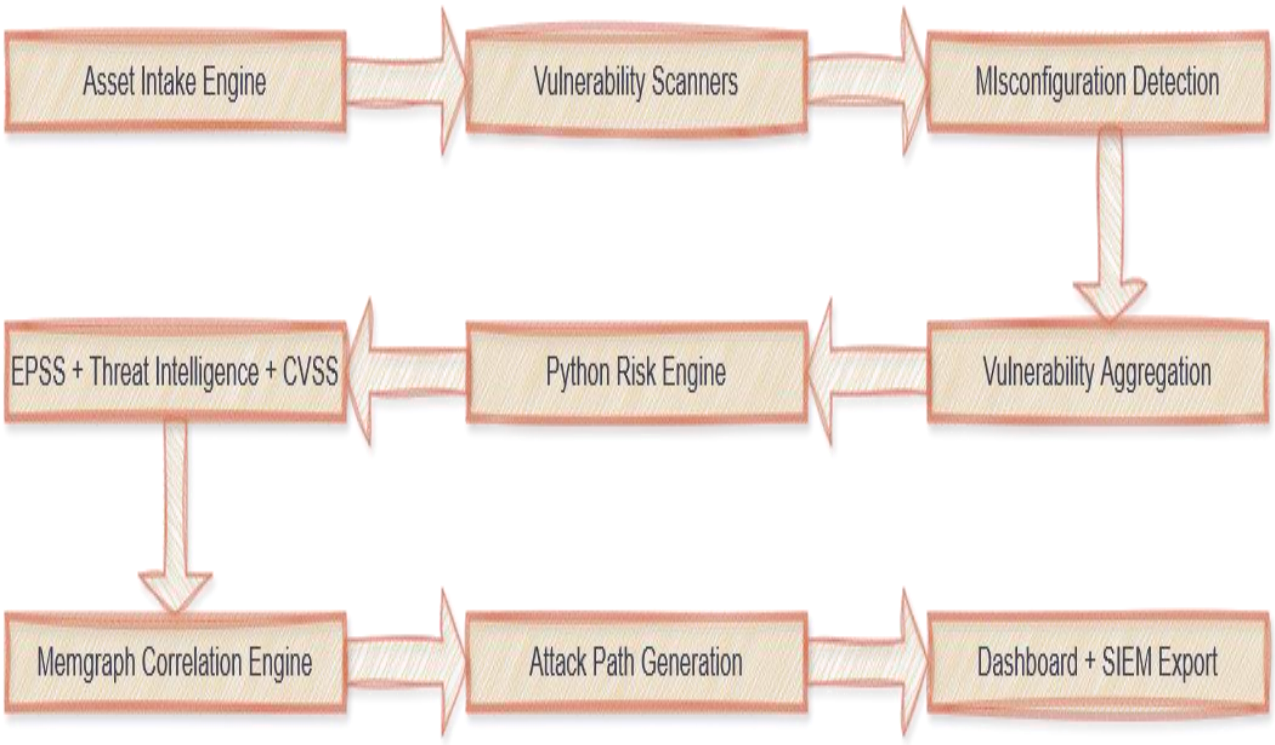
- Too many false positive
- Duplicate findings
- Lack of exploitation context
- No attack path visibility
- Poor prioritization
- Manual triage workload
- Lack of real-time automation

The Threatgraph implementation solves these problems using automation, graph correlation, threat intelligence, and intelligent risk scoring.

3. System Objectives

The platform is designed to:
1.Perform continuous vulnerability scanning
2.Detect cloud and infrastructure misconfiguration
3.Centralize findings from multiple scanners
4.Enrich vulnerabilities with real-world threat intelligence
5.Calculate contextual risk scores
6.Identify attack paths and lateral movement possibilities
7.Automate alerting and prioritization
8. Export and analyze graph relationships using Memgraph

4. High-Level Architecture



5. Technology Stack

Component	Technology
Backend API	Python FastAPI
Workflow Automation	Beehive
Vulnerability Scanner	Nuclei
Deep Vulnerability Scanner	OpenVAS
Container Security	Trivy
Threat Intelligence	OpenCTI
Vulnerability Management	DefectDojo
Risk Engine	Python
Graph Database	Memgraph
Queue System	RabbitMQ
Relational Database	PostgreSQL

6. Why These Technologies Were Selected?

Nuclei
Selected because:

- ✓ Fast internet-scale scanning
- ✓ Lightweight architecture
- ✓ Template-based vulnerability detection
- ✓ Excellent automation support
- ✓ Open-source and highly customizable

OpenVAS
Selected because:

- ✓ Deep authenticated scanning
- ✓ Enterprise-style vulnerability assessment
- ✓ Strong CVE coverage
- ✓ Compliance checks

Trivy
Selected because:
- ✓ Cloud native security scanning
- ✓ Container vulnerability detection
- ✓ Secret scanning support
- ✓ Kubernetes compatibility

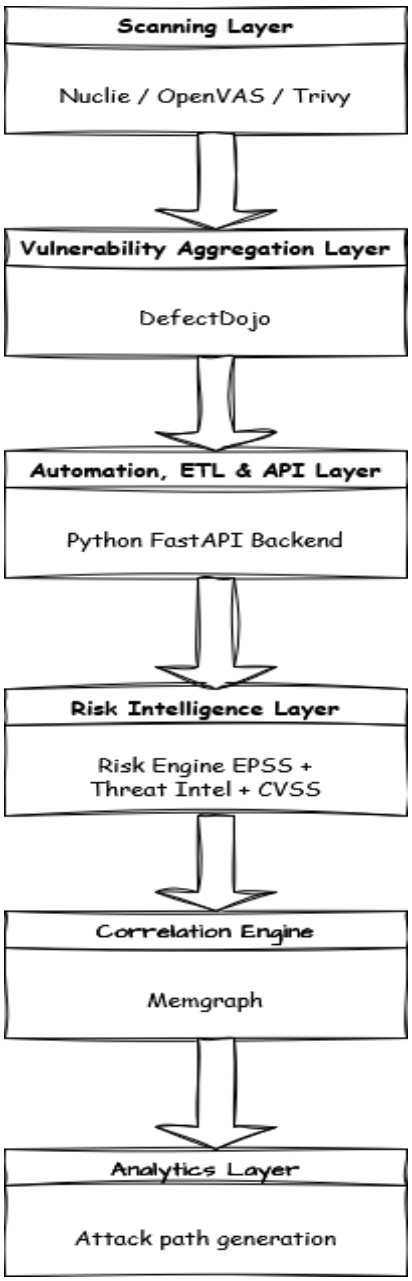
DefectDojo
Selected because:
- ✓ Centralized vulnerability management
- ✓ Deduplication support
- ✓ REST API integrations
- ✓ Enterprise vulnerability workflows

Memgraph
Selected because:
- ✓ Native graph relationships
- ✓ Fast attack path traversal
- ✓ Ideal for lateral movement analysis
- ✓ Better relationship modelling than relational databases

Python
Selected because:
- ✓ Excellent cybersecurity ecosystem
- ✓ Strong automation support
- ✓ API integration capabilities
- ✓ AI and machine learning support

OpenCTI
Selected because:
- ✓ Provides threat intelligence enrichment
- ✓ Links CVEs with:
  - Threat actors
  - Malware
  - Campaigns

7. Workflow



Phase 1 – Vulnerability Discovery

Different scanners operate with defined responsibilities:

- Nuclei → Web vulnerabilities (external attack surface)
- OpenVAS → Internal infrastructure vulnerabilities

- Trivy → Containers, images, and Kubernetes

Phase 2 – Vulnerability Aggregation

All findings are centralized into DefectDojo, which performs:

- Normalization
- Deduplication
- Severity mapping
- Lifecycle management

Phase 3 – Risk Enrichment

The Python Risk Engine retrieves data via API and enriches vulnerabilities using:

- CVSS score
- EPSS probability
- Threat intelligence (OpenCTI)
- Exploit availability
- Asset criticality

Phase 4 – Graph Correlation

Enriched data is stored in Memgraph.
Graph Model:
(:Asset)-[:HAS_VULNERABILITY]->(:Vulnerability)
(:Vulnerability)-[:HAS_EXPLOIT]->(:Exploit)
(:User)-[:HAS_ACCESS]->(:Asset)
(:Asset)-[:EXPOSED_TO]->(:Internet)

Phase 5 – Attack Path Generation

Graph traversal algorithms identify:

- Lateral movement paths
- Chained vulnerabilities
- Privilege escalation routes
- High-risk exposure paths

8. Risk Engine Logic

Final Risk Score = (CVSS/10 × 0.35) + (EPSS × 0.30) + (Asset Criticality × 0.20) + (Exploit Availability × 0.15)

Why:
CVSS alone is insufficient
EPSS predicts exploit likelihood
Asset criticality affects business impact
Exploit intelligence improves accuracy

9. Attack Path Generation

- How nodes represent assets/users/vulnerabilities
- How edges represent relationships
- How graph traversal identifies lateral movement paths

Example attack chain:

Internet-facing server
↓
Remote Code Execution (RCE)
↓
Credential Theft
↓
Privilege Escalation
↓
Database Access

10. Workflow Orchestration

Beehive Responsibilities:

- Scheduling scans
- Triggering pipelines
- Managing job execution
- Handling retries and failures

11. Output & Integration

The platform exports results to:

- SIEM Systems (e.g., Splunk, Elastic SIEM)
- Dashboards for visualization
- Graph-based attack path views